

Assignment IV: Portfolio Replication — Detailed Execution Plan

Data Overview

- **Source:** Mega/ folder containing ~500 individual stock CSVs (e.g. AAPL.csv, MSFT.csv) + ^GSPC.csv for the benchmark
- **Format:** Daily OHLCV per file, ticker = filename
- **Date range:** Jan 2020 – Feb 2026
- **Splits:**
 - **Train:** Jan 2020 – Dec 2022
 - **Validation (in-sample test):** Jan 2023 – Jun 2025
 - **Holdout:** Jul 2025 – Dec 2025 ← never touched until final evaluation

Team Roles

Person	Role
P1	Data Pipeline & Infrastructure
P2	Method 1 — Lasso
P3	Method 2 — Autoencoder
P4	Evaluation Harness & Visualizations

Day 1

P1 — Data Ingestion & Cleaning

Goal: Produce clean, shared return matrices that everyone else depends on.
Done by EOD.

Steps:

1. Load all CSVs:

```
import os, pandas as pd

data_dir = "Mega/"
prices = {}
for f in os.listdir(data_dir):
    if f.endswith(".csv") and f != "^GSPC.csv":
        ticker = f.replace(".csv", "")
        df = pd.read_csv(f"{data_dir}/{f}", index_col="Date", parse_dates=True)
        prices[ticker] = df["Close"]
```

```

price_df = pd.DataFrame(prices).sort_index()

```

2. **Compute daily log returns:**

```

returns = np.log(price_df / price_df.shift(1)).dropna(how="all")

```
3. **Handle missing data:**
 - Drop any stock that has >20% missing values over the full period
 - Forward-fill short gaps (3 days) for the rest
 - Note: some tickers (e.g. ABNB) were listed after Jan 2020 — they will have NaNs at the start. Either drop them or restrict their use to the period they exist.
4. **Load benchmark:**

```

gspc = pd.read_csv("Mega/~/GSPC.csv", index_col="Date", parse_dates=True)
bench_returns = np.log(gspc["Close"] / gspc["Close"].shift(1)).dropna()

```
5. **Align index:**

```

common_idx = returns.index.intersection(bench_returns.index)
returns = returns.loc[common_idx]
bench_returns = bench_returns.loc[common_idx]

```
6. **Train/val/holdout splits:**

```

train_end      = "2022-12-31"
val_end        = "2025-06-30"
holdout_start  = "2025-07-01"

R_train = returns.loc[:train_end]
b_train = bench_returns.loc[:train_end]

R_val    = returns.loc[train_end:val_end]
b_val    = bench_returns.loc[train_end:val_end]

R_hold   = returns.loc[holdout_start:]
b_hold   = bench_returns.loc[holdout_start:]

```
7. **Sector mapping:**
 - Use yfinance or a static GICS CSV to map each ticker → sector
 - Compute S&P 500 sector weights (equal-weight within sector, or market-cap weighted if market cap data is available)
 - Save as sector_map.csv
8. **Save artifacts:**

```

returns.to_parquet("returns_all.parquet")
bench_returns.to_frame("bench").to_parquet("bench_returns.parquet")
# share via repo or shared folder

```

Deliverable by EOD Day 1: returns_all.parquet, bench_returns.parquet, sector_map.csv

P4 — Evaluation Harness

Goal: A shared eval.py module that Persons 2 & 3 just import. Done by EOD.

```
# eval.py

import numpy as np
import pandas as pd

def tracking_error(port_returns: pd.Series, bench_returns: pd.Series) -> float:
    """Annualised tracking error."""
    active = port_returns - bench_returns
    return active.std() * np.sqrt(252)

def information_ratio(port_returns: pd.Series, bench_returns: pd.Series) -> float:
    """IR = mean(active returns) / std(active returns), annualised."""
    active = port_returns - bench_returns
    if active.std() == 0:
        return np.nan
    return (active.mean() / active.std()) * np.sqrt(252)

def portfolio_returns(weights: dict, returns_df: pd.DataFrame) -> pd.Series:
    """
    weights: {ticker: float} - should sum to 1
    returns_df: T x N DataFrame
    """
    tickers = list(weights.keys())
    w = np.array([weights[t] for t in tickers])
    w = w / w.sum() # normalise just in case
    return returns_df[tickers] @ w

def evaluate(weights: dict, R_val, b_val, R_hold, b_hold, label="") -> dict:
    """Full evaluation on both val and holdout sets."""
    p_val = portfolio_returns(weights, R_val)
    p_hold = portfolio_returns(weights, R_hold)
    return {
        "label": label,
        "k": len(weights),
        "TE_val": tracking_error(p_val, b_val),
        "IR_val": information_ratio(p_val, b_val),
        "TE_hold": tracking_error(p_hold, b_hold),
    }
```

```

        "IR_hold": information_ratio(p_hold, b_hold),
    }

```

Also set up skeleton plotting functions — P2/P3 just call these with their results:

```

def plot_cumulative(weights_dict_per_method, R, bench, title=""):
    """weights_dict_per_method: {'Lasso': {...}, 'AE': {...}}"""
    ...

def plot_te_vs_k(results_df, title="Sparsity vs Tracking Error"):
    """results_df: columns = [k, TE_val, TE_hold, method]"""
    ...

```

P2 — Lasso Setup

Read up on the formulation. The regression is:

$$b_t = \sum_i w_i * r_{\{i,t\}} + \text{epsilon}_t$$

L1 penalty on w forces most weights to zero — the nonzero ones are your selected stocks.

```

from sklearn.linear_model import LassoCV, Lasso

```

```

# Fit on train

```

```

X_train = R_train.fillna(0).values    # T x N
y_train = b_train.values

```

```

lasso_cv = LassoCV(cv=5, max_iter=10000, positive=True)
lasso_cv.fit(X_train, y_train)

```

Note: `positive=True` enforces long-only weights, which is important — you don't want a short portfolio replicating an index.

End of Day 1: have the basic pipeline running on train data.

P3 — Autoencoder Setup

Architecture to implement:

```

Input (N stocks) → Dense(128, relu) → Dense(32, relu) [bottleneck]
                  → Dense(128, relu) → Output (N stocks)

```

```

import torch

```

```

import torch.nn as nn

```

```

class ReturnAutoencoder(nn.Module):
    def __init__(self, n_stocks, latent_dim=32):

```

```

super().__init__()
self.encoder = nn.Sequential(
    nn.Linear(n_stocks, 128), nn.ReLU(),
    nn.Linear(128, latent_dim), nn.ReLU()
)
self.decoder = nn.Sequential(
    nn.Linear(latent_dim, 128), nn.ReLU(),
    nn.Linear(128, n_stocks)
)

def forward(self, x):
    return self.decoder(self.encoder(x))

```

- Input: each row is a *cross-section* of N stock returns on day t (shape: T × N)
- Train to minimise MSE reconstruction loss
- After training, compute **communality** for each stock i as the R² of reconstructing stock i's return series from the latent representation

End of Day 1: have the model training on R_train.

Day 2

P2 — Lasso: Sweep k

Lasso doesn't directly control k, but alpha (regularization strength) controls sparsity. Sweep alpha to get different k values:

```

results = []
for alpha in np.logspace(-4, -1, 40):
    lasso = Lasso(alpha=alpha, positive=True, max_iter=10000)
    lasso.fit(X_train, y_train)

    selected = {ticker: w for ticker, w in zip(R_train.columns, lasso.coef_) if w > 1e-6}
    k = len(selected)
    if k == 0:
        continue

    # Normalise weights to sum to 1
    total = sum(selected.values())
    weights = {t: w/total for t, w in selected.items()}

    res = evaluate(weights, R_val, b_val, R_hold, b_hold, label="Lasso")
    res["alpha"] = alpha
    results.append(res)

```

```
lasso_results = pd.DataFrame(results)
```

Filter to one row per k (keep best alpha per k), then hand lasso_results to P4.

P3 — Autoencoder: Stock Selection

After training, extract communality scores:

```
# Get latent representations
model.eval()
with torch.no_grad():
    Z = model.encoder(torch.tensor(R_train.fillna(0).values, dtype=torch.float32))
    R_reconstructed = model.decoder(Z).numpy()

# Communality =  $R^2$  per stock
from sklearn.metrics import r2_score
communality = {}
for i, ticker in enumerate(R_train.columns):
    communality[ticker] = r2_score(R_train.iloc[:, i].fillna(0), R_reconstructed[:, i])

# Rank stocks by communality
ranked = sorted(communality.items(), key=lambda x: -x[1])

Sweep k by taking top-k stocks by communality, then solving a QP for weights:

from scipy.optimize import minimize

def fit_weights_qp(selected_tickers, R_train, b_train):
    X = R_train[selected_tickers].fillna(0).values
    y = b_train.values
    n = len(selected_tickers)

    def objective(w):
        active = X @ w - y
        return active.var()

    constraints = [{"type": "eq", "fun": lambda w: w.sum() - 1}]
    bounds = [(0, 1)] * n # long only
    w0 = np.ones(n) / n

    res = minimize(objective, w0, bounds=bounds, constraints=constraints)
    return dict(zip(selected_tickers, res.x))

results_ae = []
for k in range(10, 101, 5):
```

```

top_k = [t for t, _ in ranked[:k]]
weights = fit_weights_qp(top_k, R_train, b_train)
res = evaluate(weights, R_val, b_val, R_hold, b_hold, label="Autoencoder")
results_ae.append(res)

ae_results = pd.DataFrame(results_ae)

```

Share `ae_results` with P4 by end of Day 2.

P4 — Preliminary Plots

Once P2 & P3 share early numbers, build: - Cumulative return overlays (k=50 portfolio vs benchmark) for both methods - Preliminary TE-vs-k line chart (will be finalized Day 3)

Day 3

Morning: Go/No-Go on GA

If P3 has spare bandwidth, implement GA on top of the existing `fit_weights_qp` function (the fitness function is already written).

GA sketch:

```

import random

def ga_select(R_train, b_train, k, pop_size=50, n_gen=100):
    tickers = list(R_train.columns)
    N = len(tickers)

    # Initialise population: each individual = list of k ticker indices
    population = [random.sample(range(N), k) for _ in range(pop_size)]

    def fitness(individual):
        selected = [tickers[i] for i in individual]
        w = fit_weights_qp(selected, R_train, b_train)
        p = portfolio_returns(w, R_train)
        return -tracking_error(p, b_train) # maximise negative TE

    for gen in range(n_gen):
        scored = sorted(population, key=fitness, reverse=True)
        elite = scored[:pop_size//2]

        # Crossover
        children = []

```

```

while len(children) < pop_size//2:
    p1, p2 = random.sample(elite, 2)
    cut = k // 2
    child = list(set(p1[:cut] + p2[cut:]))
    # Fix cardinality
    while len(child) < k:
        child.append(random.choice([i for i in range(N) if i not in child]))
    child = child[:k]
    children.append(child)

# Mutation: swap one stock
for ind in children:
    if random.random() < 0.1:
        swap_out = random.randint(0, k-1)
        swap_in = random.choice([i for i in range(N) if i not in ind])
        ind[swap_out] = swap_in

population = elite + children

best = sorted(population, key=fitness, reverse=True)[0]
return [tickers[i] for i in best]

```

Cutoff: midday Day 3. If results aren't ready by then, drop GA entirely.

P1 — Sector Drift Chart (Day 3)

```

# S&P 500 true sector weights (equal-weight per stock)
sector_map = pd.read_csv("sector_map.csv", index_col="ticker")
sp500_sector_weights = sector_map["sector"].value_counts(normalize=True)

# Portfolio sector weights for k=50 Lasso portfolio
selected_tickers = list(lasso_k50_weights.keys())
portfolio_sectors = sector_map.loc[selected_tickers, "sector"].value_counts(normalize=True)

# Align and plot
drift_df = pd.DataFrame({
    "S&P 500": sp500_sector_weights,
    "Portfolio (k=50)": portfolio_sectors
}).fillna(0)

drift_df.plot(kind="bar", figsize=(12,5), title="Sector Drift: k=50 vs S&P 500")

```

P4 — Final Deliverables (Day 3)

1. **TE vs k line chart** — both methods on the same axes, val and holdout curves
2. **Cumulative return chart** — k=50 portfolio (both methods) vs $\hat{\text{GSPC}}$
3. **Sector drift bar chart** (from P1)
4. **Results table:**

Method	k	TE (Val)	TE (Holdout)	IR (Val)	IR (Holdout)
Lasso	50	...	...	...	...
Autoencoder	50	...	...	...	...
GA (if included)	50	...	...	...	...

5. Package everything into a single clean `notebook.ipynb`

File Structure

```
Assignment4/
  Mega/                                # raw CSVs (read-only)
  data/
    returns_all.parquet # P1 output
    bench_returns.parquet
    sector_map.csv
  eval.py                 # P4 shared harness
  method_lasso.py         # P2
  method_autoencoder.py   # P3
  method_ga.py            # P3 optional
  plots.py               # P4
  notebook.ipynb         # final assembly
```

Key Risks & Mitigations

Risk	Mitigation
Tickers with post-2020 listings (ABNB, etc.) causing NaN issues	Drop or restrict to available date range in P1
Lasso selecting k=0 at high alpha	Add a floor: skip alphas where $k < 5$
Autoencoder overfitting on train	Use dropout + early stopping on a val split within train

Risk	Mitigation
QP solver failing to converge	Fall back to OLS with non-negativity clipping
GA too slow	Reduce pop_size to 20, n_gen to 50 as a quick run
P1 delays blocking everyone	P2/P3 can start with a 50-stock random subset as a smoke test